

Big Data Computing

Master's of Science in Computer Science

Sapienza University of Rome

2026/2027

Gabriele Tolomei

tolomei@di.uniroma1.it

Module 1: Big Data Infrastructure

Lecture 1.3: Apache Spark & Dataflow Graph Architectures

Philosophy of Spark

"The first rule of distributed computing is: don't move the data. The second rule is: don't lose the data either — even when you have to recompute it."

— Adapted from distributed systems folklore

MapReduce solved survivability. Spark asks: now that we know how to survive failure, can we also stay fast — by keeping data in memory and remembering how to rebuild it, rather than replicating it?

| Lecture Roadmap

This lecture generalizes the two-stage MapReduce model into something far more flexible and memory-centric.

Specifically, we will study:

1. **Beyond Two-Stage Pipelines** — why MapReduce's rigidity becomes a bottleneck
2. **Resilient Distributed Datasets (RDDs)** — the core abstraction
3. **Transformations & Actions** — the RDD programming interface
4. **Lazy Evaluation & the Execution DAG** — building before running
5. **Narrow vs. Wide Dependencies** — what determines execution structure
6. **Lineage Tracking & Fault Tolerance** — surviving failure without replication
7. **Synthesis** — putting the whole execution model together

Let's see how Spark rebuilds MapReduce's guarantees on top of memory, not just disk.

| Why This Matters: From MapReduce's Limits to Spark

Lecture 1.2 ended by identifying real limitations of the MapReduce model:

- Every job's intermediate data is written to **disk** between Map and Reduce — costly when repeated across many chained jobs.
- The model is rigidly **two-stage** — complex pipelines require gluing together many separate MapReduce jobs.
- Iterative workloads (graph algorithms, machine learning training loops) pay this disk and re-launch cost **on every iteration**.

Apache Spark, introduced via the Resilient Distributed Datasets (RDD) paper (Zaharia et al., NSDI 2012), was designed directly to address these three limitations.

§ 1 — Beyond Two-Stage Pipelines

| The MapReduce Ceiling, Revisited

Recall the three limitations identified at the end of Lecture 1.2:

- **Disk-heavy:** intermediate data between Map and Reduce always touches local disk.
- **Rigid two-stage structure:** every job is exactly Map → Shuffle → Reduce, no more, no less.
- **High latency:** favors large batch throughput, not fast iterative or interactive workloads.

Each of these stems from the same root cause: MapReduce was designed around disk-resident data and a fixed pipeline shape.

| Multi-Stage Pipelines as Chained MapReduce Jobs

Real-world analytics often require more than one Map-Reduce step:

```
Job 1: Map → Reduce → (write output to HDFS)  
Job 2: Map → Reduce → (read Job 1's output, write new output to HDFS)  
Job 3: Map → Reduce → (read Job 2's output, ...)
```

- Each job is launched, scheduled, and torn down **independently**.
- Every intermediate result is written to, and re-read from, the distributed file system.

| Why Chaining MapReduce Jobs Is Expensive

Each additional job in a chain pays the full cost of a standalone MapReduce job:

- A full **write** to HDFS/GFS (with replication) at the end of every job.
- A full **read** from HDFS/GFS at the start of the next job.
- A fresh **shuffle** phase, even if the same keys were already grouped in a previous step.

For a 10-stage iterative pipeline, this means 10 full disk round-trips and 10 shuffle phases — even though only the very first input and very last output truly need durable storage.

| The General Need: Arbitrary Computation Graphs

What real pipelines actually need is the ability to express **arbitrary chains of operations**, not just one fixed Map → Reduce shape:

- Filter, then transform, then join with another dataset, then aggregate, then filter again, and so on.
- The "shape" of computation is a general graph of operations, not a fixed two-stage template.

This motivates moving from "two hardcoded stages" to "an arbitrary, programmer-defined graph of stages."

| Introducing DAGs, Informally

A **Directed Acyclic Graph (DAG)** is a natural way to represent such a computation:

- **Nodes** represent operations (or the datasets they produce).
- **Directed edges** represent data dependencies — "this operation's output feeds that operation's input."
- **Acyclic:** no cycles — computation always flows forward, never back on itself.

A MapReduce job is, in this view, simply a DAG with exactly two stages. Spark generalizes this to DAGs of arbitrary shape and depth.

DAG vs. Two-Stage Pipeline: Diagram

MapReduce (fixed shape):



Spark (arbitrary DAG):



MapReduce always looks like the left diagram. Spark can express the right diagram — or any other shape a programmer's logic requires.

| What "Memory-Centric" Means

Beyond generalizing the graph shape, Spark's second major shift is keeping data **in memory** across operations, whenever possible:

- Intermediate results between operations are held in RAM, not written to disk by default.
- Data is only written to durable storage when the programmer explicitly requests it, or when memory is insufficient.

This directly attacks the "disk-heavy" limitation of MapReduce identified in Lecture 1.2.

| Why Memory Changes the Cost Model

Recall from Lecture 1.1's memory hierarchy discussion: RAM access is roughly **100x faster** than disk access for equivalent data volumes.

- Re-reading intermediate data from memory across pipeline stages avoids the disk round-trip entirely.
- For **iterative** algorithms — where the same dataset is scanned repeatedly — this difference compounds with every iteration.
- This is the central performance argument of the original RDD paper: keep working datasets resident in memory across an entire multi-step or iterative computation.

| Spark's Founding Motivation (NSDI Paper Context)

The RDD paper (Zaharia et al., 2012) frames Spark's motivation around workloads that MapReduce served poorly:

- Existing dataflow systems (like MapReduce) lacked an efficient way to **reuse data across multiple operations**.
- The paper's proposed answer: a new abstraction — the **Resilient Distributed Dataset (RDD)** — that is both memory-resident and fault-tolerant, without relying on replication.

| Iterative Algorithms as the Motivating Workload

A primary use case driving Spark's design: algorithms that repeatedly scan the **same** dataset across many iterations:

- Iterative graph algorithms (e.g., PageRank-style computations).
- Iterative machine learning training loops (e.g., gradient descent over a fixed training set).
- Each iteration in MapReduce would mean: re-read input from disk, re-run a full Map-Reduce job, write output, repeat.

| Interactive and Exploratory Queries as a Second Motivation

A second key motivation: enabling fast, **interactive** querying of a dataset already loaded into a cluster's memory:

- A data analyst exploring a dataset wants successive queries to run in seconds, not minutes.
- If the dataset is cached in memory once, repeated queries against it avoid repeated disk I/O entirely.
- MapReduce's batch-job model is ill-suited to this kind of rapid, exploratory interaction.

| Positioning Spark Relative to MapReduce

Aspect	MapReduce	Spark
Pipeline shape	Fixed two-stage (Map → Reduce)	Arbitrary DAG of operations
Intermediate data	Written to disk between stages	Held in memory when possible
Reuse across iterations	Re-read from disk each time	Cached in memory across iterations
Fault tolerance	Re-execution of failed tasks	Lineage-based recomputation (Section 6)
Best fit	Large one-pass batch jobs	Iterative, interactive, multi-step pipelines

✓ Checkpoint 1 — Motivation for Spark

Let's test our basic understanding:

1. What three limitations of MapReduce does Spark directly address?
2. Why is chaining multiple MapReduce jobs expensive, even when each individual job is efficient?
3. How does a DAG generalize the fixed two-stage MapReduce pipeline?
4. Why do iterative algorithms benefit disproportionately from memory-centric execution?

Write down your thoughts before moving to Section 2.

§ 2 — Resilient Distributed Datasets (RDDs)

| What Is an RDD, Informally?

A **Resilient Distributed Dataset (RDD)** is Spark's core data abstraction:

- A collection of elements, **partitioned** across the machines in a cluster.
- **Immutable:** once created, an RDD's contents never change.
- **Resilient:** can be automatically recovered after a partition is lost, without relying on replication (mechanism covered in Section 6).

Every dataset a Spark program manipulates — whether loaded from storage or derived from another dataset — is represented as an RDD.

| **RDD as a Distributed, Immutable Collection**

Conceptually, an RDD behaves like a large, distributed version of a simple in-memory collection (e.g., a list):

- You can apply operations to it (covered in Section 3) to produce new RDDs.
- You cannot modify an existing RDD's elements directly — any "change" produces a brand-new RDD.

This conceptual simplicity — "just a big distributed list" — is deliberate, hiding most distributed-systems complexity from the programmer, much like MapReduce's map/reduce functions did.

| Why Immutability Matters

RDD immutability is not an arbitrary restriction — it is foundational to Spark's design:

- Since an RDD never changes after creation, **how it was created is a complete and permanent description of its contents.**
- This recorded "how it was created" history is exactly what enables lineage-based fault tolerance (Section 6) — no need to replicate the data itself.
- Immutability also avoids a wide class of concurrency bugs that arise from shared mutable state across machines.

| RDD Partitions: The Unit of Parallelism

Like the storage abstractions from Lecture 1.1 and the task abstractions from Lecture 1.2, RDDs are split into **partitions**:

- Each partition is a subset of the RDD's data, residing on (or processed by) one machine at a time.
- Operations on an RDD are applied **independently, in parallel**, to each partition.
- The number of partitions determines the maximum degree of parallelism for operations on that RDD.

| Partitions vs. Blocks: Callback to Lecture 1.1

RDD partitions are conceptually related to, but distinct from, the storage blocks studied in Lecture 1.1:

- When an RDD is created from a file in HDFS/GFS, its partitions typically correspond directly to the file's underlying **blocks**.
- However, partitions can also be **repartitioned** explicitly, or created from non-file sources (e.g., in-memory collections) — partitioning is an RDD-level concept, independent of any specific storage layout.

| How RDDs Relate to Underlying Storage

An RDD is not itself a storage system — it is a **computational abstraction** layered on top of existing storage:

- An RDD can be created by reading from HDFS, GFS, or other storage backends.
- An RDD can also be created from data already in memory (e.g., a driver program's local collection, distributed out to the cluster).
- The RDD abstraction is deliberately agnostic to where its initial data physically comes from.

Creating RDDs: From Storage, From Collections

Two primary ways an RDD comes into existence:

```
# From distributed storage (e.g., HDFS)
rdd = sc.textFile("hdfs://path/to/data")

# From an existing in-memory collection
rdd = sc.parallelize([1, 2, 3, 4, 5])
```

- `textFile`: partitions are derived from the underlying file's blocks.
- `parallelize`: the driver's local collection is split into partitions and distributed to the cluster.

| RDD as a Programming Abstraction, Not a Physical Structure

It is important to distinguish what an RDD **represents** from what physically exists at any given moment:

- An RDD is a logical handle describing "a dataset, partitioned this way, derived from this computation."
- The actual data may or may not be materialized in memory at any given time — this depends on lazy evaluation (Section 4) and caching decisions (this section, shortly).

| The "Resilient" in RDD: Preview of Fault Tolerance via Lineage

The "R" in RDD stands for **Resilient** — but notably, this resilience does **not** come from replicating data across machines (unlike GFS/HDFS blocks in Lecture 1.1):

- Instead, Spark records **how** each RDD was derived from its parent RDD(s) — its **lineage**.
- If a partition is lost (e.g., a machine fails), Spark can **recompute** just that partition using the recorded lineage, rather than reading a replica.

Full mechanics covered in Section 6 — for now, simply note that this is a fundamentally different fault-tolerance strategy than replication.

| RDD Interface Overview

Conceptually, the RDD interface exposes:

- **Transformations** — operations that derive a new RDD from an existing one (Section 3).
- **Actions** — operations that trigger actual computation and return a result to the driver program or write output (Section 3).
- **Persistence controls** — explicit instructions to cache an RDD in memory (or disk) for reuse (covered shortly in this section).

| Why RDDs Generalize the MapReduce Input Model

Recall from Lecture 1.2 that MapReduce's input was a fixed set of blocks, processed by exactly one Map phase:

- An RDD can be created from blocks just like MapReduce input — but unlike MapReduce, it can then be transformed through an **arbitrary chain** of further operations.
- There is no fixed limit of "one Map phase, one Reduce phase" — an RDD can be transformed, filtered, joined, and aggregated in any sequence the programmer defines.

| RDD vs. a Regular Distributed Array: What's Different

An RDD resembles a distributed array or list, but with key differences:

- **No random in-place updates:** you cannot mutate element `i` directly — only derive new RDDs via transformations.
- **Lineage-aware:** the system tracks exactly how each RDD was produced, which a plain distributed array does not.
- **Lazily materialized:** an RDD's contents may not actually exist in memory until an action forces computation (Section 4).

| In-Memory Caching of RDDs

A central performance feature: an RDD can be explicitly **cached** (persisted) in memory across multiple uses:

```
rdd = sc.textFile("hdfs://path/to/data")  
rdd.cache() # mark this RDD for in-memory persistence
```

- Without caching, Spark may need to recompute an RDD from scratch (via lineage) every time it is reused.
- With caching, the computed partitions are kept in memory, available for fast reuse by later operations — directly enabling the iterative and interactive workloads from Section 1.

| Persistence Levels: Memory, Disk, Mixed

Spark allows different **storage levels** for cached RDDs, balancing speed against capacity:

Storage Level	Behavior
Memory only	Fastest; partitions that don't fit are simply recomputed when needed
Memory and disk	Spills excess partitions to local disk rather than discarding them
Disk only	Avoids memory pressure entirely, at the cost of disk-speed access

The right choice depends on dataset size relative to available cluster memory.

✓ Checkpoint 2 — RDD Fundamentals

Let's test our understanding:

1. What does it mean for an RDD to be immutable, and why does this matter for fault tolerance?
2. How do RDD partitions relate to, but differ from, the storage blocks studied in Lecture 1.1?
3. Why is RDD fault tolerance described as fundamentally different from GFS/HDFS's replication-based approach?
4. What is the purpose of explicitly caching an RDD in memory?

Discuss with your colleague or write down your answers.

§ 3 — Transformations & Actions

| Two Categories of RDD Operations

Every operation a Spark program performs on an RDD falls into exactly one of two categories:

- **Transformations:** derive a new RDD from an existing one.
- **Actions:** trigger computation and produce a result outside the RDD abstraction (returned to the driver, or written to storage).

This split is the foundation for lazy evaluation, covered in Section 4.

Transformations: Definition and Examples

A **transformation** takes one (or more) RDDs and produces a **new** RDD, describing a derived dataset:

```
map(f)           # apply f to every element
filter(f)        # keep only elements where f returns true
flatMap(f)       # apply f, flattening the resulting lists
distinct()       # remove duplicate elements
union(other)     # combine with another RDD
```

- Transformations are deterministic functions over data, much like the Map function from Lecture 1.2.

Transformations Return New RDDs

A key consequence of immutability (Section 2): calling a transformation **never modifies** the original RDD.

```
rdd1 = sc.textFile("data.txt")
rdd2 = rdd1.filter(lambda line: "error" in line)
# rdd1 is unchanged; rdd2 is a new RDD describing the filtered result
```

- `rdd1` remains exactly as it was — `rdd2` is a separate, newly described dataset.
- Chains of transformations simply produce a chain of new RDD descriptions, one after another.

| Actions: Definition and Examples

An **action** triggers Spark to actually compute results, either returning them to the driver program or writing them to storage:

```
count()           # return the number of elements
collect()         # return all elements to the driver
reduce(f)         # aggregate elements using f
saveAsTextFile(path) # write the RDD's contents to storage
```

Until an action is called, no actual data processing has necessarily happened yet — covered fully in Section 4.

| Actions Trigger Computation

The contrast with transformations is direct:

- Calling a **transformation** simply records "here is a new RDD, derived this way" — no data is processed yet.
- Calling an **action** is what actually causes Spark to execute the necessary computation across the cluster.

This distinction — describing computation vs. triggering it — is the basis of Spark's lazy evaluation model, explored in depth next.

| Why This Split Exists: Preview of Laziness

Separating "describing" from "triggering" computation enables a key optimization opportunity:

- Spark can see the **entire chain** of transformations before any of them actually run.
- This allows Spark to optimize the whole pipeline as a unit — rather than executing each operation eagerly, one at a time, as soon as it's called.

Full mechanics of this optimization are the subject of Section 4.

| Worked Example: Word Count Expressed as RDD Transformations

Revisiting Lecture 1.2's canonical word count example, now expressed using RDDs:

```
lines = sc.textFile("hdfs://documents/")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
```

- `flatMap`, `map`, and `reduceByKey` are all **transformations** — none of them have triggered computation yet.

| Tracing Transformations Without Execution

At this point in the word count example, **nothing has actually been computed**:

- `lines`, `words`, `pairs`, and `counts` are all just RDD descriptions — chained transformations describing *how* to compute the result, not the result itself.
- No data has been read from HDFS, no `flatMap` has run, no counting has happened.

This may seem surprising — it is precisely the lazy evaluation behavior explored next.

| The Action That Triggers It

Only when an action is called does Spark actually execute the chain:

```
result = counts.collect() # ACTION: triggers execution of the entire chain
```

- This single line causes Spark to: read input from HDFS, run `flatMap`, run `map`, run `reduceByKey` (including any necessary shuffle), and finally collect results to the driver.
- All of the previously-defined transformations execute together, as part of this one triggered computation.

Common Transformations Table

Transformation	Effect
<code>map(f)</code>	Apply <code>f</code> to each element
<code>filter(f)</code>	Keep elements where <code>f</code> is true
<code>flatMap(f)</code>	Apply <code>f</code> , flatten resulting lists
<code>groupByKey()</code>	Group values by key
<code>reduceByKey(f)</code>	Aggregate values per key using <code>f</code>
<code>join(other)</code>	Join two key-value RDDs by key

| Common Actions Table

Action	Effect
<code>count()</code>	Return number of elements
<code>collect()</code>	Return all elements to driver
<code>take(n)</code>	Return first <code>n</code> elements
<code>reduce(f)</code>	Aggregate all elements using <code>f</code>
<code>saveAsTextFile(path)</code>	Write RDD contents to storage

| Transformations Are Not "Free" — They Still Need Eventual Execution

Laziness does not mean transformations have no cost — it means their cost is **deferred**:

- Every transformation still needs to eventually run, as part of whatever action ultimately triggers the chain.
- What changes is *when* the work happens and *how much visibility* Spark has into the full pipeline before deciding how to run it.

| How This Differs From MapReduce's Eager Two-Stage Execution

Aspect	MapReduce	Spark (RDDs)
When does work happen?	Immediately, as each job runs	Deferred until an action is called
Visibility into the pipeline	One job at a time, in isolation	Entire chain of transformations visible before execution
Chaining multiple steps	Separate jobs, separate disk round-trips	Single chain, optimized and executed together

✓ Checkpoint 3 — Transformations & Actions

Let's test our understanding:

1. What is the fundamental difference between a transformation and an action?
2. In the word count example, why has no actual computation happened until `collect()` is called?
3. Why does separating "describing" computation from "triggering" it create an optimization opportunity?
4. Give one example each of a transformation and an action not used in the word count example.

Review your answers before moving on.

§ 4 — Lazy Evaluation & the Execution DAG

| What Laziness Means Here

Lazy evaluation is the principle that transformations are recorded as a plan, not executed immediately:

- Calling `.map()`, `.filter()`, or any other transformation only updates Spark's internal *description* of the computation.
- No partition is actually read, transformed, or written until an action forces the issue.

This is the mechanism underlying everything observed in the word count trace from Section 3.

| Why Delay Execution?

Delaying execution is not merely a quirk — it is a deliberate design choice with real benefits:

- Spark can see the **entire** pipeline of transformations before committing to a specific execution strategy.
- This visibility enables **optimizations across operations** that would be impossible if each transformation executed eagerly, in isolation.
- It also avoids wasted work: if an action only needs *part* of the data (e.g., `take(5)`), Spark can sometimes avoid computing more than necessary.

| Building a Logical Execution Plan

As transformations are chained, Spark builds an internal **logical execution plan**:

- Each RDD records a reference to its **parent RDD(s)** and the transformation that produced it.
- This forms a chain (or more generally, a graph) of RDDs, each pointing back to how it was derived.

This structure is exactly the lineage introduced conceptually in Section 2 — now seen in its role as an execution plan.

| The DAG of Transformations

The chain of RDDs and their dependencies naturally forms a **Directed Acyclic Graph**:

- Each RDD is a node; each transformation is a directed edge from parent RDD(s) to the new RDD.
- Operations that combine multiple RDDs (e.g., `join`, `union`) create nodes with multiple incoming edges.

This is the same DAG concept introduced informally in Section 1 — now grounded in the concrete RDD/transformation structure.

| DAG Diagram: Chained Transformations

Continuing the word count example from Section 3:

```
lines —flatMap→ words —map→ pairs —reduceByKey→ counts  
(textFile)
```

- Each arrow is a transformation; each box is an RDD.
- This entire diagram exists in Spark's memory **before** `collect()` is ever called — it is the logical plan, not yet executed.

| Optimization Opportunities Laziness Enables

Having full visibility into the DAG before execution lets Spark apply several optimizations:

- **Operator fusion / pipelining:** chain compatible operations together to avoid unnecessary intermediate materialization (next slide).
- **Avoiding unneeded computation:** skip parts of the DAG not required to produce the requested action's result.
- **Choosing physical execution strategy:** decide how to group operations into stages based on the dependency structure (Section 5).

| Operator Fusion / Pipelining Within a Stage

When a sequence of transformations can be applied to the same data element without needing to wait for siblings or other partitions, Spark can **pipeline** them together:

```
words.map(f).filter(g).map(h)
```

- Rather than materializing the full intermediate result after `map(f)`, then again after `filter(g)`, Spark can apply `f`, then `g`, then `h` to each element **in a single pass**, per partition.
- This avoids unnecessary memory overhead and extra passes over the data.

| Avoiding Unnecessary Intermediate Materialization

Pipelining directly reduces the cost of multi-step pipelines compared to MapReduce's job-chaining approach (Section 1):

- In chained MapReduce jobs, each job's *complete* output had to be materialized (written to HDFS) before the next job could start.
- With pipelining, intermediate results for chained, compatible transformations may never need to be fully materialized at all — data flows through several operations in a single pass.

| Triggering Execution: From Logical DAG to Physical Jobs

When an action is finally called, Spark converts the logical DAG into something it can actually run on the cluster:

1. The **DAG scheduler** examines the full chain of dependencies leading to the requested action.
2. It divides the DAG into a set of **stages** (next concept, fully explained once dependencies are covered in Section 5).
3. Each stage is further divided into **tasks**, one per partition, scheduled across the cluster — conceptually similar to MapReduce's task scheduling from Lecture 1.2.

| Stages: How the DAG Is Broken Into Stages

A **stage** is a group of operations that can be executed together, without requiring data to be shuffled across the network in between:

- Operations within a single stage are pipelined together (as just described).
- A new stage begins whenever the DAG requires a **shuffle** — data must be redistributed across partitions before the next operation can proceed.

Why shuffles force a stage boundary is explained precisely in Section 5, once narrow and wide dependencies are introduced.

| Why Stage Boundaries Exist: A Preview

Stage boundaries are not arbitrary — they exist at exactly the points where an operation cannot be computed using only **local**, per-partition data:

- Operations like `map` or `filter` can be pipelined freely — they need no data from other partitions.
- Operations like `reduceByKey` or `join` require data to be regrouped *across* partitions — exactly like the shuffle phase from MapReduce (Lecture 1.2).

Section 5 formalizes this distinction as narrow vs. wide dependencies.

Worked Example: Building and Triggering a Multi-Step DAG

Extending the word count example with an additional step — filtering out rare words after counting:

```
lines = sc.textFile("hdfs://documents/")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
frequent = counts.filter(lambda kv: kv[1] > 100)
result = frequent.collect() # ACTION triggers the entire chain
```

Five chained transformations, executed as a single coordinated DAG when `collect()` is finally called.

Comparing This to MapReduce's Fixed Two-Stage Shape

Aspect	MapReduce (Lecture 1.2)	Spark DAG (this lecture)
Pipeline depth	Exactly Map then Reduce	Arbitrary number of chained operations
Intermediate storage	Disk, between every job	Pipelined in memory where possible
Multi-step pipelines	Multiple separate jobs	A single DAG, optimized as a whole
Where shuffle occurs	Always, once, between Map and Reduce	Only at wide-dependency boundaries (Section 5)

| Checkpoint 4 — Lazy Evaluation & DAGs

Let's test our understanding:

1. What does it mean for Spark transformations to be "lazy," and what triggers actual execution?
2. What is operator fusion/pipelining, and why does it reduce unnecessary intermediate materialization?
3. What is a "stage," and what causes a new stage to begin?
4. How does Spark's DAG-based execution differ from MapReduce's fixed two-stage execution model?

Consolidate your answers before proceeding.

§ 5 — Narrow vs. Wide Dependencies

| Dependencies Between RDDs: What They Capture

A **dependency** describes the relationship between a child RDD's partitions and the partition(s) of its parent RDD(s) that it depends on:

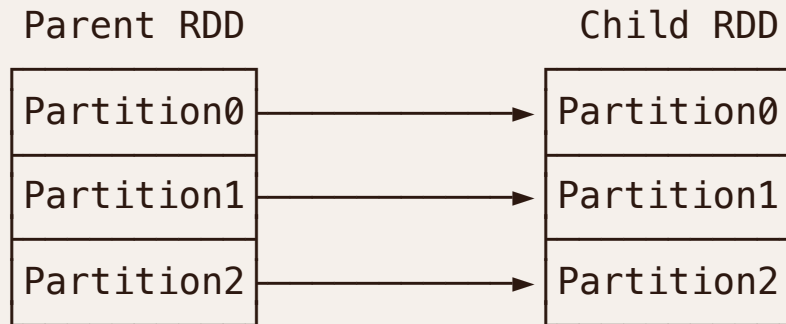
- This is the precise, formal version of the "transformation history" introduced informally as lineage in Section 2.
- The *type* of dependency — narrow or wide — determines how expensive a transformation is to execute, and whether it forces a new stage.

| Narrow Dependency: Defined

A dependency is **narrow** when each partition of the child RDD depends on a **small, fixed number of partitions** (typically exactly one) of the parent RDD:

- `map`, `filter`, and `flatMap` are all narrow: each output partition is computed purely from the corresponding single input partition.
- No data needs to move between partitions on different machines.

Narrow Dependency: Diagram



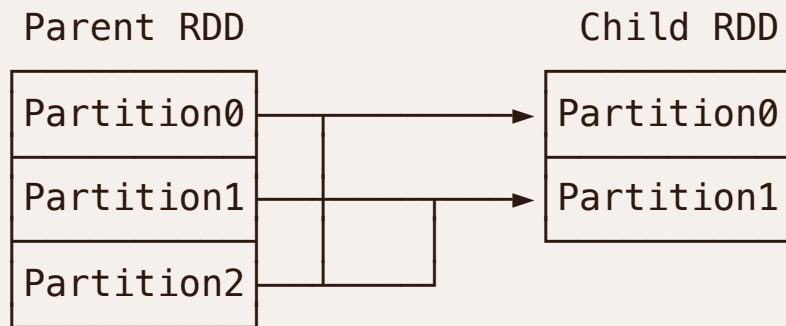
Each child partition depends on exactly one parent partition — entirely local, no network transfer required.

| Wide Dependency: Defined

A dependency is **wide** when a child RDD's partition may depend on data from **multiple, potentially all**, partitions of the parent RDD:

- `groupByKey`, `reduceByKey`, and `join` are all wide: producing any single output partition may require data originally scattered across every input partition.
- This is structurally identical to the all-to-all data movement problem in MapReduce's shuffle (Lecture 1.2, Section 3).

Wide Dependency: Diagram



Each child partition may depend on data scattered across multiple parent partitions — requiring network data movement (a shuffle).

| Why the Distinction Matters for Execution

The narrow/wide distinction is the precise mechanism behind the stage boundaries introduced in Section 4:

- Narrow dependencies allow safe **pipelining** — a chain of narrow transformations can be computed entirely within one partition, on one machine, in one pass.
- Wide dependencies require a **shuffle** — data must be redistributed across the network before the next operation can proceed, exactly as in MapReduce.

| Narrow Dependencies Enable Pipelining Within a Stage

Because narrow transformations never need data from other partitions, Spark can chain any number of them into a single pipelined stage:

```
data.map(f1).filter(f2).map(f3)
```

- All three operations apply to each partition independently, in a single pass, without any cross-partition coordination.
- This is exactly the pipelining behavior introduced in Section 4 — now we see precisely *why* it's possible: every step here is a narrow dependency.

| Wide Dependencies Force a Shuffle: Callback to Lecture 1.2

A wide dependency like `reduceByKey` cannot be computed without first regrouping data by key across the cluster:

- This regrouping step is, structurally, the **same shuffle operation** studied in Lecture 1.2's MapReduce treatment — partitioning, network transfer, and grouping by key.
- Spark does not avoid the cost of shuffling; it simply confines that cost to exactly the points in the DAG where it is unavoidable, rather than imposing it after every job as MapReduce's chained-job model does.

| Stage Boundaries = Wide Dependency Boundaries

We can now state precisely what determines a stage boundary, completing the picture from Section 4:

- A **stage** is a maximal run of narrow-dependency transformations that can be pipelined together.
- A **new stage begins** exactly at each wide dependency, because computing it requires the shuffle's output to be available before proceeding.

| Worked Example: Identifying Narrow vs. Wide in a DAG

Returning to the extended word count DAG from Section 4:

```
lines —flatMap→ words —map→ pairs —reduceByKey→ counts —filter→ frequent  
      (narrow)      (narrow)      (wide)              (narrow)
```

- `flatMap`, `map`, and `filter` are narrow — these form pipelined segments within stages.
- `reduceByKey` is wide — this is exactly where a stage boundary (and a shuffle) occurs.

*Result: this five-step chain compiles down to just **two** physical stages.*

| Performance Implications of Wide Dependencies

Wide dependencies carry the same cost characteristics identified for MapReduce's shuffle in Lecture 1.2:

- Network bandwidth consumption proportional to the volume of data being redistributed.
- Disk I/O for writing and reading shuffle data (Spark may spill shuffle data to disk under memory pressure).
- A natural synchronization point: all partitions contributing to a wide dependency must complete before the next stage can fully proceed.

| Minimizing Wide Dependencies in Practice

Because wide dependencies are the dominant cost driver, Spark programs are often written to minimize their number and impact:

- Prefer `reduceByKey` over `groupByKey` followed by a separate aggregation, since `reduceByKey` can partially aggregate locally before shuffling (directly analogous to a MapReduce combiner, Lecture 1.2).
- Where possible, restructure pipelines to push narrow operations (filtering, mapping) **before** wide operations, reducing the volume of data that must be shuffled.

✓ Checkpoint 5 — Narrow vs. Wide Dependencies

Let's test our understanding:

1. What distinguishes a narrow dependency from a wide dependency?
2. Why do narrow dependencies allow pipelining, while wide dependencies do not?
3. In the worked DAG example, why does the chain compile down to exactly two stages?
4. How is a wide dependency in Spark related to the shuffle phase studied in MapReduce?

Make sure you understand these mechanisms before wrapping up this section.

§ 6 — Lineage Tracking & Fault Tolerance

| The Fault Tolerance Problem, Restated

As in both prior lectures, failures are assumed to be routine at cluster scale:

- A machine holding one or more RDD partitions can crash mid-computation.
- Lecture 1.1's answer: replicate data across machines (GFS/HDFS).
- Lecture 1.2's answer: re-execute failed tasks from the original input.
- Spark's answer, building on both: **recompute only the lost partitions**, using recorded lineage — without needing replicated copies of every intermediate RDD.

| Lineage: What It Is

Lineage is the recorded chain of transformations that describes exactly how an RDD's partitions were derived from their parent RDD(s):

- Every RDD remembers: which parent RDD(s) it came from, and which transformation (and partitioning function) produced it.
- This is precisely the DAG structure built up across Sections 4 and 5 — now viewed in its role as a fault-tolerance mechanism rather than an execution-planning mechanism.

| Lineage as a Record of Transformations, Not Data Copies

This is the central distinction between Spark's fault tolerance and GFS/HDFS's:

- GFS/HDFS fault tolerance: keep **multiple physical copies** of the actual data (Lecture 1.1).
- Spark fault tolerance: keep a **compact description** of how to recreate the data, and recompute it on demand.

Lineage information is tiny compared to the data it describes — a few transformation references, regardless of how large the underlying dataset is.

| How Lineage Enables Recomputation

If a partition of some RDD is lost (e.g., the machine holding it crashes), Spark can rebuild exactly that partition:

1. Identify which parent partition(s) the lost partition depends on (its recorded lineage).
2. If those parent partitions are still available (in memory, on disk, or themselves recomputable), recompute the lost partition by reapplying the relevant transformation.
3. If a parent partition is *a*lso missing, recurse further back along the lineage chain until reaching available data.

| Recomputing Lost Partitions From Lineage

Original RDD —map→ RDD A —filter→ RDD B —reduceByKey→ RDD C

- Suppose a partition of RDD B is lost (its machine crashed).
- Spark traces back: RDD B's partition depends on a known partition of RDD A.
- If that RDD A partition is still available, Spark simply **re-applies** **filter** to recreate the lost RDD B partition — no other partitions need to be touched.

| Comparing to MapReduce's Re-execution Model

Aspect	MapReduce (Lecture 1.2)	Spark (RDDs)
What is recomputed	The entire failed task (Map or Reduce)	Only the specific lost partition(s)
Recovery granularity	Whole task	Individual partition, following lineage
Multi-step pipelines	Each job's failure handled independently	A single lineage chain spans the whole pipeline

Spark's approach is a direct generalization of MapReduce's "just recompute it" philosophy — applied per-partition, across an arbitrarily long DAG.

| Comparing to GFS/HDFS's Replication Model: Callback to Lecture 1.1

Aspect	GFS/HDFS (Lecture 1.1)	Spark RDDs
Fault tolerance mechanism	Physical replication (typically 3×)	Lineage-based recomputation
Storage overhead	3× the data volume, by default	Minimal — just the transformation graph
Recovery cost	Read an existing replica (fast)	Recompute from lineage (can be slow if lineage is long)

This is a genuine trade-off, not a strict improvement — explored on the next slide.

| Why Lineage Is Cheaper Than Replicating Every RDD

Given how many intermediate RDDs a typical Spark program creates, replicating each one would be prohibitively expensive:

- A pipeline with ten chained transformations would, under a replication strategy, require storing (and keeping synchronized) ten separate replicated datasets.
- Lineage instead stores only a small, constant amount of metadata per RDD — regardless of how large the underlying dataset is — making it dramatically cheaper for the common case of long transformation chains.

| Limits of Lineage: Long Chains, Recomputation Cost

Lineage-based recovery is not free of trade-offs:

- If a lineage chain is very **long**, recomputing a lost partition near the end of the chain may require recomputing a large portion of everything before it.
- This recomputation cost grows with the **depth** of the DAG — particularly problematic for iterative algorithms where the same RDD is transformed repeatedly across many iterations.

| Checkpointing RDDs to Bound Recomputation Cost

To address long-lineage recomputation cost, Spark allows explicit **checkpointing**:

- A checkpoint writes an RDD's current data to durable, replicated storage (e.g., HDFS) — similar in spirit to GFS/HDFS's replication strategy from Lecture 1.1.
- After checkpointing, the lineage chain is effectively "reset" — future recomputation only needs to go back to the checkpoint, not all the way to the original input.

Checkpointing is a deliberate trade-off: pay an upfront storage/replication cost now, in exchange for bounded recomputation cost later.

| DAG Scheduling and Fault Recovery Together

Fault recovery integrates directly with the scheduling mechanisms from Sections 4–5:

- When a task fails (e.g., due to a machine crash), the DAG scheduler identifies exactly which RDD partition(s) were lost.
- It re-schedules new tasks to recompute just those partitions, using the same stage/dependency structure already established for normal execution.
- This means fault recovery in Spark reuses the *same* DAG machinery used for ordinary execution planning — there is no separate, special-cased recovery system.

✓ Checkpoint 6 — Lineage & Fault Tolerance

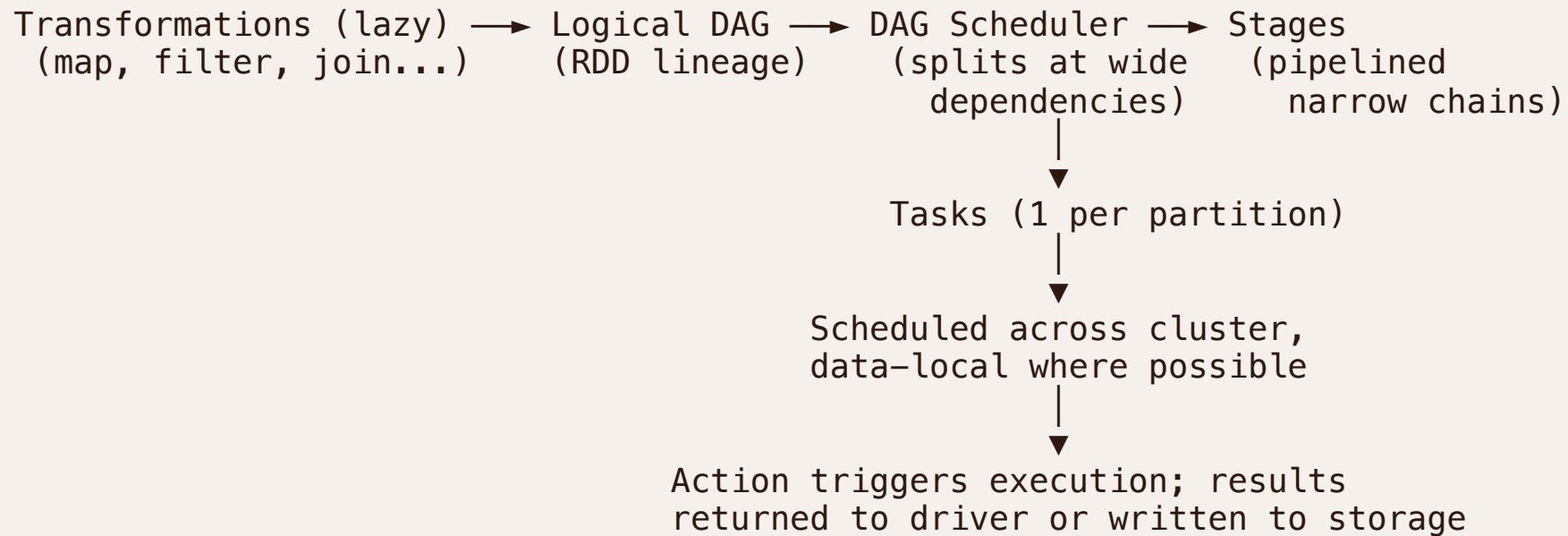
Let's test our understanding:

1. How does Spark's lineage-based fault tolerance differ fundamentally from GFS/HDFS's replication-based approach?
2. Walk through how Spark recovers a single lost RDD partition.
3. Why can long lineage chains become a liability, and how does checkpointing address this?
4. In what sense does Spark's fault-tolerance mechanism reuse the same machinery as its normal DAG execution?

Consolidate your answers before proceeding to the final section.

§ 7 — Synthesis & Wrap-up

Full Spark Execution Model Recap Diagram



Every box in this diagram was built, mechanism by mechanism, across Sections 1–6.

| **DAG Scheduler: Role**

The **DAG Scheduler** is Spark's coordinating component, conceptually analogous to the Master/JobTracker from Lecture 1.2:

- Receives the logical RDD lineage graph once an action is called.
- Determines stage boundaries based on narrow vs. wide dependencies (Section 5).
- Submits each stage's tasks for execution only once that stage's input dependencies are satisfied.

| From RDD Lineage Graph to Physical Task Scheduling

The DAG Scheduler's output is a concrete, executable plan:

1. Walk the lineage graph backward from the action's target RDD.
2. Group narrow-dependency chains into pipelined stages.
3. Order stages so that each one's wide-dependency inputs (shuffle outputs) are ready before it runs.
4. Within each stage, create one task per partition, ready for scheduling onto cluster machines.

| Data Locality in Spark's Scheduler: Callback

Just as in Lecture 1.1 (GFS/HDFS) and Lecture 1.2 (MapReduce), Spark's task scheduler prefers to run tasks **near** the data they need:

- For an RDD's first stage (reading from HDFS/GFS), tasks are scheduled on machines holding the corresponding blocks, exactly as in MapReduce's Map task scheduling.
- For later stages, Spark prefers machines where the required (possibly cached) parent partitions already reside in memory, avoiding unnecessary data movement.

| Strengths of the Spark/RDD Model

Reflecting on what this model achieves, relative to MapReduce:

- **Generality:** arbitrary DAGs of operations, not a fixed two-stage shape.
- **Performance for iterative/interactive workloads:** in-memory caching avoids repeated disk round-trips.
- **Efficient fault tolerance:** lineage-based recomputation avoids the storage overhead of full replication for every intermediate dataset.
- **Optimizable execution:** full pipeline visibility before execution enables pipelining and avoids unnecessary work.

| Remaining Limitations

The RDD model, while a major advance over MapReduce, still has its own limitations:

- The core RDD API is relatively **low-level** — expressing complex relational-style queries directly via transformations can be verbose.
- RDDs alone provide no built-in understanding of **structured data** (schemas, columns) that could enable further optimization.
- The original batch-oriented model does not natively address **continuous, unbounded streaming data**.

| Modern Data Infrastructure & Streaming Frameworks

More recently, other frameworks have moved beyond the batch-oriented model entirely:

- **Continuous stream processing** (Apache Flink) — processing unbounded data as it arrives.
- **Dynamic task graphs for distributed AI** (Ray).
- **Unified Lakehouse storage abstractions** (Delta Lake) — bridging the gap between data lakes and structured tables.

| Summary: Key Concepts of This Lecture

1. **The abstraction:** RDDs — immutable, partitioned, lineage-tracked distributed datasets.
2. **The programming interface:** transformations (lazy, build the DAG) and actions (trigger execution).
3. **The execution structure:** narrow dependencies enable pipelining; wide dependencies force shuffles and stage boundaries.
4. **Fault tolerance:** lineage-based recomputation, with checkpointing to bound recovery cost — a fundamentally different strategy from replication.

| Key Terms Glossary

Term	Meaning
RDD	Immutable, partitioned, distributed dataset with tracked lineage
Transformation	Lazy operation producing a new RDD (e.g., map, filter)
Action	Operation that triggers execution and returns/writes a result
Narrow dependency	Child partition depends on one parent partition; pipelined
Wide dependency	Child partition depends on many parent partitions; requires shuffle
Lineage	Recorded transformation history enabling recomputation

| Lecture 1.3 Complete

Recommended Reading

- Zaharia, M., et al. (2012). *Resilient Distributed Datasets*. NSDI.
- Course reference materials on Spark execution internals

Contact & Q&A

- **Lecturer:** Gabriele Tolomei
- **Email:** tolomei@di.uniroma1.it
- **Office Hours:** Drop me a message, and we'll arrange a time to meet, either in person or online! 😊

Thank you for your attention!